

lab4_N-Grams.py

Spiegazione dettagliata, riga per riga, del codice del Laboratorio 4

"Modeling social media and literary language with N-grams"

Corso di Tecnologie del Linguaggio Naturale — Prof. Daniele Radicioni, UniTO

File analizzato: TLN/Parte2Radicioni/lab4_N-Grams.py (479 righe)

Obiettivo del laboratorio: confrontare, tramite modelli n-gram, il linguaggio dei social media (Twitter) con quello letterario (il romanzo *Emma* di Jane Austen), usando stima MLE e smoothing Laplace, generazione di testo, perplexity e classificazione per dominio.

Come leggere questo documento: ogni sezione riprende un blocco del codice (nell'ordine in cui appare nel file) e lo espone in una tabella a tre colonne: numero di riga, codice sorgente, spiegazione. Le spiegazioni non descrivono solo *cosa* fa la riga, ma soprattutto **perché** è stata scritta così, con particolare attenzione alle scelte di design non ovvie.

0. Intestazione, import e setup (righe 1–29)

Questo blocco prepara l'ambiente: importa tutte le librerie necessarie, configura l'encoding dell'output e scarica le risorse NLTK (corpora e tokenizer) che verranno usate nel resto dello script.

Riga	Codice	Spiegazione
1–2	<pre># LAB 4 - Modeling social media and literary language with N-grams # TLN - Daniele Radicioni, UNITO</pre>	Commenti descrittivi: titolo dell'esperimento e riferimento al corso. Non hanno effetto sull'esecuzione, servono da documentazione per chi legge il file.
4	<pre>import nltk</pre>	Libreria principale per l'elaborazione del linguaggio naturale: fornisce sia i corpora di testo (Twitter, Gutenberg) sia gli strumenti di tokenizzazione e i modelli linguistici n-gram usati più avanti.
5	<pre>import re</pre>	Modulo per le espressioni regolari, usato nelle funzioni di pulizia del testo per rimuovere URL, menzioni, hashtag e caratteri non alfabetici.
6	<pre>import math</pre>	Serve esclusivamente per <code>math.log()</code> , usato nel calcolo manuale della log-likelihood (sezione classificatore).
7	<pre>from collections import Counter</pre>	<code>Counter</code> è un dizionario specializzato per il conteggio: viene usato per contare le occorrenze di bigrammi/trigrammi in modo efficiente e per ottenere subito i più frequenti con <code>.most_common()</code> , evitando di scrivere a mano la logica di conteggio e ordinamento.
8	<pre>from nltk.tokenize import word_tokenize, sent_tokenize</pre>	<code>word_tokenize</code> spezza una stringa in token (parole/punteggiatura); <code>sent_tokenize</code> spezza un testo lungo in frasi. Quest'ultimo è indispensabile per il corpus letterario, che arriva come un unico blocco di testo continuo.
9	<pre>from nltk import ngrams</pre>	Funzione che, data una sequenza di token e un valore <code>n</code> , genera tutte le tuple di <code>n</code> token consecutivi. È il mattone base per costruire bigrammi (<code>n=2</code>) e trigrammi (<code>n=3</code>) da contare con <code>Counter</code> .
10	<pre>from nltk.lm import MLE, Laplace</pre>	Le due classi di modello linguistico n-gram usate nel laboratorio: <code>MLE</code> (Maximum Likelihood Estimation, stima "pura" per frequenza relativa, senza smoothing) e <code>Laplace</code> (stessa idea ma con smoothing add-one, che assegna probabilità non nulla anche agli n-grammi mai osservati). Il confronto fra i due è uno degli obiettivi didattici del laboratorio.

11	<pre>from nltk.lm.preprocessing import padded_everygram_pipeline, pad_both_ends</pre>	<code>padded_everygram_pipeline</code> prepara automaticamente i dati di addestramento (aggiunge i marcatori di inizio/fine sequenza <code><s> / </s></code> e genera gli n-grammi da 1 a n); <code>pad_both_ends</code> fa la stessa cosa di padding ma per una singola sequenza, usata più avanti per calcolare la perplexity di frasi di test.
12	<pre>import numpy as np</pre>	Usato solo nella sezione grafici, per generare le posizioni numeriche delle barre affiancate (<code>np.arange</code>).
13-15	<pre>import matplotlib matplotlib.use('Agg') import matplotlib.pyplot as plt</pre>	Perché 'Agg' : il backend "Agg" è un renderer che scrive direttamente su file raster, senza aprire una finestra grafica. È fondamentale impostarlo <i>prima</i> di importare <code>pyplot</code> perché lo script viene eseguito in modo non interattivo (terminale/CI, senza display): con il backend interattivo di default, la chiamata a <code>plt.savefig()</code> più avanti potrebbe fallire o tentare di aprire una GUI inesistente.
16-17	<pre>import sys import os</pre>	Moduli di sistema: <code>sys</code> per riconfigurare l'encoding dello stdout, <code>os</code> per calcolare il percorso di salvataggio del grafico in modo indipendente dalla directory corrente.
18	<pre>sys.stdout.reconfigure(encoding='utf-8')</pre>	Perché : su Windows la console usa spesso una codepage legacy (es. cp1252) che non contiene caratteri come →, ✓, ✕ usati più avanti nei <code>print()</code> . Senza questa riga, il primo <code>print</code> con uno di questi simboli solleverebbe <code>UnicodeEncodeError</code> e il programma terminerebbe.
21-25	<pre>nltk.download('twitter_samples') nltk.download('gutenberg') nltk.download('punkt') nltk.download('punkt_tab') nltk.download('stopwords')</pre>	Scarica (se non già presenti in cache locale) le risorse NLTK necessarie: il corpus di tweet campione, il corpus Gutenberg (testi letterari di pubblico dominio), i modelli di tokenizzazione <code>punkt / punkt_tab</code> (usati internamente da <code>word_tokenize / sent_tokenize</code>) e la lista di stopwords inglesi. Perché farlo qui : rende lo script eseguibile "a freddo" su qualunque macchina, senza richiedere un setup manuale separato; le chiamate sono idempotenti (se la risorsa è già presente, non ricaricano nulla).
27	<pre>from nltk.corpus import twitter_samples, gutenberg, stopwords</pre>	Import degli oggetti "corpus reader" veri e propri, fatto <i>dopo</i> i download per garantire che le risorse siano disponibili quando questi reader verranno effettivamente usati (l'import in sé è lazy, ma l'accesso ai dati fallirebbe se il download non fosse avvenuto).
29	<pre>print("Import completati!")</pre>	Messaggio di controllo: conferma visivamente che la fase di setup (import + download) è terminata senza errori prima di procedere con l'elaborazione dei dati.

1. Caricamento dei due corpora (righe 31–40)

Vengono caricati i due dataset che rappresentano i due "domini linguistici" da confrontare: un insieme di tweet (linguaggio breve e informale) e il romanzo *Emma* di Jane Austen (linguaggio lungo e formale).

Riga	Codice	Spiegazione
31	<pre># due domini a confronto: social (breve/informale) vs letterario (lungo/formale)</pre>	Commento che dichiara esplicitamente l'ipotesi sperimentale del laboratorio: i due corpora sono scelti apposta per essere agli antipodi stilistici, così le differenze nei modelli n-gram risulteranno marcate e facili da osservare.
32–34	<pre>tweets_pos = twitter_samples.strings('positive_t weets.json') tweets_neg = twitter_samples.strings('negative_t weets.json') tweets_raw = tweets_pos + tweets_neg</pre>	Il corpus <code>twitter_samples</code> di NLTK contiene due file JSON pre-etichettati per sentiment (5000 tweet positivi, 5000 negativi). <code>.strings()</code> restituisce il testo grezzo di ciascun tweet come lista di stringhe. Perché unirli: per questo esperimento il sentiment non è rilevante — serve solo un campione ampio e vario di linguaggio "da social" — quindi le due liste vengono concatenate in un unico dataset da 10.000 tweet.
36	<pre>testo_letterario_raw = gutenberg.raw('austen-emma.txt')</pre>	<code>gutenberg.raw()</code> restituisce l'intero testo del romanzo come un'unica stringa (a differenza di <code>.strings()</code> per Twitter, qui non c'è già una suddivisione in "unità" testuali separate: il romanzo dovrà essere spezzato in frasi più avanti con <code>sent_tokenize</code>).
38–40	<pre>print(f"Tweet caricati: {len(tweets_raw)}") print(f"Esempio positivo: {tweets_pos[0]}") print(f"Esempio negativo: {tweets_neg[0]}")</pre>	Stampe di controllo ("sanity check"): permettono di verificare a occhio che il caricamento sia andato a buon fine e che il formato dei dati sia quello atteso, prima di investire tempo nell'elaborazione successiva.

2. Pulizia del testo (righe 42–62)

Il testo grezzo contiene elementi (URL, menzioni, punteggiatura, maiuscole) che, se non normalizzati, gonfierebbero artificialmente il numero di token distinti e distorcerebbero le frequenze degli n-grammi. Questa sezione definisce due funzioni di pulizia, una per dominio, perché i due tipi di testo hanno rumore diverso.

Riga	Codice	Spiegazione
42-43	<pre>def pulisci_tweet(testo): """Rimuove URL, menzioni e hashtag, che distorcerebbero le frequenze degli n-grammi."""</pre>	Funzione dedicata alla pulizia dei tweet. Il docstring motiva la scelta: URL, @menzioni e hashtag sono elementi tipici di Twitter che non fanno parte del "linguaggio" in senso stretto — un URL diverso in ogni tweet creerebbe migliaia di token unici e spuri, abbassando artificialmente la ripetibilità degli n-grammi.
44	<pre> testo = re.sub(r'http\S+', '', testo)</pre>	Rimuove gli URL: <code>http</code> seguito da uno o più caratteri non di spaziatura (<code>\S+</code>), quindi cattura link <code>http/https</code> interi fino al primo spazio.
45	<pre> testo = re.sub(r'@\w+', '', testo)</pre>	Rimuove le menzioni (es. <code>@utente</code>): sono identificatori di account, non parole del linguaggio naturale, quindi vanno eliminate interamente (non solo la <code>@</code>).
46	<pre> testo = re.sub(r'#', '', testo)</pre>	Rimuove solo il simbolo <code>#</code> , mantenendo la parola che segue. Perché: a differenza delle menzioni, il testo di un hashtag (es. "loveit" in <code>#loveit</code>) è spesso una parola o frase con contenuto semantico utile; si scarta solo il marcatore sintattico.
47	<pre> testo = re.sub(r'^[a-zA-Z\s]', '', testo)</pre>	Elimina tutto ciò che non è una lettera dell'alfabeto inglese o uno spazio: numeri, punteggiatura, emoji, caratteri Unicode speciali. Questo semplifica drasticamente il vocabolario (niente "!!!", niente emoji diverse trattate come token distinti) a costo di perdere informazione (es. punteggiatura enfatica, emoticon testuali).
48	<pre> testo = testo.lower().strip()</pre>	Minuscolizza (così "Love" e "love" diventano lo stesso token, evitando di raddoppiare inutilmente il vocabolario e diluire le frequenze) e rimuove spazi bianchi iniziali/finali residui dalle sostituzioni precedenti.
49	<pre> return testo</pre>	Restituisce la stringa pulita. Esempio reale (eseguendo lo script): il primo tweet del corpus, tracciato riga per riga attraverso <code>pulisci_tweet</code> : Dopo la riga Contenuto della stringa

ingresso	#FollowFriday @France_Inte @PKuchly57 @Milipol_Paris for being top engaged members in my community this week :)
44 (URL)	invariato – non c'è nessun URL in questo tweet
45 (@menzioni)	#FollowFriday for being top engaged members in my community this week :) (<i>le tre menzioni sono sparite, restano gli spazi dove c'erano</i>)
46 (#)	FollowFriday for being top engaged members in my community this week :)
47 (non-alfa)	FollowFriday for being top engaged members in my community this week (<i>anche ":" è sparita</i>)
48 (lower+strip)	followfriday for being top engaged members in my community this week

Nota come restino **spazi multipli consecutivi** dove prima c'erano le menzioni: `re.sub` le sostituisce con stringa vuota, non con un singolo spazio, e `.strip()` rimuove solo gli spazi a inizio/fine stringa, non quelli interni. Questo non è un problema per `word_tokenize` (che ignora gli spazi ripetuti), ma spiega perché la stampa di controllo alla riga 62 mostra quegli spazi doppi nell'output reale dello script.

```
51-54 def pulisci_letterario(testo):
      testo = re.sub(r'^a-zA-Z\s',
'', testo)
      testo = testo.lower().strip()
      return testo
```

Versione più leggera della pulizia, senza le fasi per URL/menzioni/hashtag: il testo di un romanzo dell'Ottocento non contiene questi elementi, quindi applicare quei passaggi sarebbe codice morto. Restano solo la rimozione di caratteri non alfabetici (qui elimina anche la punteggiatura come virgole e punti) e la minuscolizzazione.

56	<pre> tweets_puliti = [pulisci_tweet(t) for t in tweets_raw]</pre>	Applica la pulizia a ogni tweet tramite list comprehension, producendo la lista parallela dei tweet ripuliti.
57	<pre> tweets_puliti = [t for t in tweets_puliti if len(t) > 0] # scarta i tweet svuotati dalla pulizia (es. solo URL/menzioni)</pre>	Perché: alcuni tweet sono composti solo da un link o da una menzione (es. "@utente http://..."); dopo la pulizia diventano stringhe vuote. Se non vengono scartati, più avanti <code>word_tokenize("")</code> produrrebbe una sequenza vuota che inquinerebbe il conteggio delle sequenze di addestramento senza portare informazione utile.
58	<pre> testo_lett_pulito = pulisci_letterario(testo_letterario _raw)</pre>	Pulisce l'intero romanzo in un colpo solo (qui non serve ancora la suddivisione in frasi, che avverrà più avanti a partire dal testo <i>grezzo</i>, non da questo).
60-62	<pre> print("\n=== PULIZIA ===") print(f"Tweet PRIMA: {tweets_raw[0]}") print(f"Tweet DOPO: {tweets_puliti[0]}")</pre>	Stampa di controllo che mostra un esempio prima/dopo, utile per verificare visivamente che le regex si comportino come previsto.

3. Tokenizzazione (righe 64–81)

Da qui in poi il testo viene trasformato in liste di token (parole). Vengono create **due rappresentazioni parallele**: un flusso unico di token per le statistiche aggregate (vocabolario, conteggi globali) e delle sequenze separate per unità testuale (tweet singolo / frase singola), indispensabili per addestrare correttamente i modelli n-gram.

Riga	Codice	Spiegazione
64	<pre>testo_tweet_unito = ' ' .join(tweets_puliti)</pre>	Concatena tutti i tweet puliti in un'unica grande stringa separata da spazi, utile solo per calcoli aggregati (token totali, vocabolario) sull'intero corpus Twitter, non per l'addestramento del modello (dove i confini tra tweet contano, vedi più sotto).
65–66	<pre>tokens_tweet = word_tokenize(testo_tweet_unito) tokens_lett = word_tokenize(testo_lett_pulito)</pre>	Tokenizza i due blob di testo in liste piatte di parole, usate per le statistiche generali (conteggio token, vocabolario, bigrammi/trigrammi "globali" nella sezione successiva).
67–68	<pre>vocab_tweet_set = set(tokens_tweet) vocab_lett_set = set(tokens_lett)</pre>	Converte le liste di token in insiemi (<code>set</code>) per ottenere il vocabolario, cioè le parole <i>uniche</i> di ciascun dominio, eliminando i duplicati automaticamente (i <code>set</code> non ammettono ripetizioni).
70–71	<pre># sequenze separate (un tweet/una frase = una sequenza) per l'addestramento: servono # a non far attraversare i bigrammi tra un tweet/frase e il successivo</pre>	Motivazione centrale di questa sezione: se si tokenizzasse tutto il corpus come un unico flusso continuo, un modello a bigrammi genererebbe anche la coppia (ultima parola del tweet N, prima parola del tweet N+1), che è una relazione sintattica <i>falsa</i> — le due parole non sono mai state scritte in sequenza da un autore. Per evitarlo, ogni tweet/frase deve essere trattato come una sequenza indipendente, con i propri marcatori di inizio/fine. Esempio concreto: supponiamo due tweet consecutivi nella lista <code>tweets_puliti</code>: "so happy today" seguito da "work was awful". • Con <code>tokens_tweet</code> (flusso unico, riga 65): i due tweet vengono concatenati in un'unica lista <code>[... , "so", "happy", "today", "work", "was", "awful", ...]</code>. Un estrattore di bigrammi su questa lista genererebbe anche la coppia <code>("today", "work")</code>, che non è mai stata scritta da nessuno — è solo un artefatto di aver incollato due tweet scorrelati. • Con <code>sequenze_tweet</code> (righe 72, usata per l'addestramento): i due tweet restano due liste separate, <code>["so", "happy", "today"]</code> e <code>["work", "was", "awful"]</code>. <code>padded_everygram_pipeline</code> (sezione 6) aggiunge

		poi <code><s> / </s></code> a ciascuna separatamente, ottenendo bigrammi come <code>("today", "</s>")</code> e <code>("<s>", "work")</code> — corretti, perché rispettano il vero confine fra i due tweet.
72	<pre>sequenze_tweet = [word_tokenize(t) for t in tweets_puliti]</pre>	Tokenizza <i>ogni tweet singolarmente</i> (non il blob unito), producendo una lista di liste: una sotto-lista di token per ciascun tweet. È questa struttura, e non <code>tokens_tweet</code>, che verrà usata per addestrare i modelli.
73	<pre>frasi_lett_raw = sent_tokenize(testo_letterario_raw)</pre>	Spezza il romanzo in frasi usando il testo grezzo (non quello già ripulito). Perché il testo grezzo e non quello pulito: <code>sent_tokenize</code> si basa sulla punteggiatura (punti, punti interrogativi, maiuscole) per riconoscere i confini di frase; <code>pulisci_letterario</code> rimuove proprio quella punteggiatura, quindi applicarla prima renderebbe impossibile individuare correttamente le frasi.
74	<pre>sequenze_lett = [word_tokenize(pulisci_letterario(f)) for f in frasi_lett_raw]</pre>	Per ciascuna frase individuata al passo precedente, applica ora la pulizia (rimozione punteggiatura, minuscolo) e poi la tokenizzazione. L'ordine delle operazioni è quindi: <i>segmenta in frasi</i> → <i>pulisci ogni frase</i> → <i>tokenizza ogni frase</i>, mentre per Twitter era <i>pulisci ogni tweet</i> → <i>tokenizza</i> (Twitter ha già un confine naturale, il singolo tweet, che non dipende dalla punteggiatura).
75	<pre>sequenze_lett = [s for s in sequenze_lett if s]</pre>	Scarta le sotto-liste vuote, che si generano quando una "frase" individuata da <code>sent_tokenize</code> era in realtà composta solo da punteggiatura/numeri (es. capitoli numerati), analogamente al filtro fatto per i tweet vuoti alla riga 57.
77-81	<pre>print("\n=== TOKENIZZAZIONE ===") print(f"Token Twitter: {len(tokens_tweet)}") print(f"Token Letterario: {len(tokens_lett)}") print(f"Parole uniche Twitter: {len(vocab_tweet_set)}") print(f"Parole uniche Letterario: {len(vocab_lett_set)}")</pre>	Stampa le statistiche dimensionali di base dei due corpora: numero totale di token e numero di parole distinte, un primo indicatore quantitativo della loro diversità.

4. Stopword e bigrammi "di contenuto" (righe 83–98)

Vengono estratti i bigrammi più frequenti dopo aver rimosso le stopwords (parole funzionali come "the", "is", "and"), per far emergere i temi ricorrenti di ciascun dominio invece delle sole strutture grammaticali.

Riga	Codice	Spiegazione
83	<pre>stop = set(stopwords.words('english'))</pre>	Carica la lista di stopwords inglesi di NLTK e la converte in <code>set</code> per avere un test di appartenenza (<code>in</code>) a tempo costante $O(1)$ invece che $O(n)$ come su una lista, importante perché verrà controllata per ogni singolo token dei due corpora.
85–86	<pre>tokens_tweet_filtrati = [t for t in tokens_tweet if t not in stop] tokens_lett_filtrati = [t for t in tokens_lett if t not in stop]</pre>	Filtra via le stopwords da entrambi i flussi di token, producendo versioni "di solo contenuto" del testo.
88	<pre># senza stopwords: piu informativi sui temi ricorrenti</pre>	Motivazione: bigrammi come ("i", "love") o ("of", "the") sono frequentissimi ma poco informativi sul <i>contenuto</i> ; rimuovendo le stopwords emergono invece coppie di parole piene (nomi, verbi, aggettivi) che rivelano i temi tipici del dominio.
89–90	<pre>bi_tweet_filtrati = Counter(ngrams(tokens_tweet_filtrat i, 2)) bi_lett_filtrati = Counter(ngrams(tokens_lett_filtrati , 2))</pre>	Genera tutti i bigrammi (coppie di token consecutivi) dai token già filtrati (senza stopwords) e li conta con <code>Counter</code> . Problema metodologico reale, non solo un'approssimazione: alle righe 85–86 il filtro delle stopwords avviene <i>prima</i> di formare le coppie. Questo causa due distorsioni: (1) il flusso resta concatenato senza confini tra un tweet/frase e il successivo (come alle righe 70–71); (2) più insidiosamente, rimuovere le stopwords prima di accoppiare le parole rende adiacenti parole che nel testo originale <i>non lo erano</i> , perché separate da una o più stopwords — generando bigrammi mai scritti da nessuno. Esempio: "i went to the store" → dopo il filtro diventa "went store" → si conta il bigramma ("went", "store"), inesistente nel testo reale. Il modo corretto sarebbe estrarre prima i bigrammi dal testo completo e scartare poi solo le coppie in cui <i>entrambe</i> le parole sono stopwords.
92–98	<pre>print("\n=== BIGRAMMI SENZA STOPWORDS ===") print("\nTWITTER (top 10):") for coppia, count in bi_tweet_filtrati.most_common(10): print(f" {str(coppia):<35} → {count} volte")</pre>	<code>.most_common(10)</code> restituisce i 10 bigrammi con frequenza più alta come lista di coppie (bigramma, conteggio), già ordinati. Il formato <code>{str(coppia):<35}</code> allinea a sinistra su 35 caratteri per ottenere colonne leggibili in output testuale.

```
print("\nLETTERARIO (top 10):")
for coppia, count in
bi_lett_filtrati.most_common(10):
    print(f" {str(coppia):<35} →
{count} volte")
```

5. Bigrammi e trigrammi "con" stopwords (righe 100–120)

La stessa estrazione viene ripetuta sul testo integrale (stopword incluse) e anche a livello di trigrammi, per confrontare stavolta lo **stile sintattico** (che dipende proprio dalle parole funzionali) piuttosto che i temi.

Riga	Codice	Spiegazione
100	# stessa estrazione sul testo integrale, per confrontare lo stile sintattico	A differenza della sezione precedente, qui l'obiettivo non è più il contenuto tematico ma le strutture grammaticali ricorrenti (es. "i love", "i cant" per Twitter vs "it was", "she had" per il registro letterario), che sono proprio le combinazioni di parole funzionali che prima erano state escluse.
101–104	<pre> bi_tweet = Counter(ngrams(tokens_tweet, 2)) bi_lett = Counter(ngrams(tokens_lett, 2)) tri_tweet = Counter(ngrams(tokens_tweet, 3)) tri_lett = Counter(ngrams(tokens_lett, 3)) </pre>	Stessa logica di conteggio della sezione 4, ma su <code>tokens_tweet / tokens_lett</code> (non filtrati) e sia per <code>n=2</code> (bigrammi) sia per <code>n=3</code> (trigrammi), per osservare come la specificità cresce all'aumentare della lunghezza dell' <code>n</code> -gramma.
106–112	<pre> print("\n=== BIGRAMMI CON STOPWORDS (top 10) ===") print("TWITTER:") for coppia, count in bi_tweet.most_common(10): print(f" {str(coppia):<30} → {count} volte") print("\nLETTERARIO:") for coppia, count in bi_lett.most_common(10): print(f" {str(coppia):<30} → {count} volte") </pre>	Stampa i 10 bigrammi (stopword incluse) più frequenti per dominio, stesso schema della sezione precedente.
114–120	<pre> print("\n=== TRIGRAMMI (top 10) ===") print("TWITTER:") for t, c in tri_tweet.most_common(10): print(f" {str(t):<42} → {c} volte") print("\nLETTERARIO:") for t, c in tri_lett.most_common(10): print(f" {str(t):<42} → {c} volte") </pre>	Stessa stampa ma per i trigrammi; la larghezza di formattazione (42 invece di 30) è più ampia perché le tuple a 3 elementi producono stringhe più lunghe di quelle a 2.

6. Addestramento dei modelli n-gram: MLE e Laplace (righe 122–143)

Qui si passa dalla semplice conta di frequenze a dei veri e propri **modelli linguistici probabilistici**. Vengono addestrati quattro modelli in totale: MLE e Laplace, ciascuno sia sul dominio Twitter sia su quello letterario, così da poter confrontare domini diversi e tecniche di stima diverse.

Riga	Codice	Spiegazione								
123	<pre>n = 2 # ordine del modello: n=2 -> bigrammi, P(parola 1 parola di contesto)</pre>	Fissa l'ordine dei modelli principali a 2 (bigrammi): ogni parola viene predetta condizionandola su una sola parola di contesto precedente. È un compromesso tra semplicità/robustezza statistica (pochi dati necessari per stimare bene le probabilità) e capacità del modello di catturare dipendenze locali.								
126	<pre>train_tw_mle, vocab_tw_mle = padded_everygram_pipeline(n, sequenze_tweet)</pre>	Prende le sequenze <i>separate</i> per tweet (non il flusso unico) e, per ciascuna, aggiunge i marcatori <code><s></code> / <code></s></code> agli estremi, poi genera tutti gli "everygram" (n-grammi da 1 a n=2) necessari per l'addestramento. Restituisce due generatori: <code>train_tw_mle</code> (i dati di training) e <code>vocab_tw_mle</code> (il vocabolario, incluso il padding). Usare le sequenze separate qui è essenziale per lo stesso motivo spiegato alla riga 70: evita bigrammi spuri a cavallo tra due tweet. Esempio reale (eseguendo <code>padded_everygram_pipeline(2, ["i", "love", "you"])</code> isolatamente, cioè con una sola sequenza giocattolo di 3 token): <table><tr><th>Cosa</th><th>Valore prodotto</th></tr><tr><td>sequenza originale</td><td><code>['i', 'love', 'you']</code></td></tr><tr><td>everygram generati (1-grammi + 2-grammi, con padding)</td><td><code>('<s>',)</code> <code>('<s>', 'i')</code> <code>('i',)</code> <code>('i', 'love')</code> <code>('love',)</code> <code>('love', 'you')</code> <code>('you',)</code> <code>('you', '</s>')</code> <code>('</s>',)</code></td></tr><tr><td>vocabolario</td><td><code>['<s>', 'i', 'love', 'you', '</s>']</code></td></tr></table> Si vede che per una sequenza di 3 parole vengono prodotti 9 n-grammi: i 3 unigrammi originali, i 2 bigrammi interni ("i love", "love you"), più i 2 marcatori di padding come unigrammi a sé (<code><s></code>, <code></s></code>) e i 2 bigrammi di confine (<code><s> i</code> e <code>you </s></code>) — sono proprio questi ultimi due bigrammi di confine che permettono al modello di imparare quali parole tendono ad aprire o chiudere una frase/tweet. Con 10.000 tweet reali, questo stesso procedimento viene ripetuto e	Cosa	Valore prodotto	sequenza originale	<code>['i', 'love', 'you']</code>	everygram generati (1-grammi + 2-grammi, con padding)	<code>('<s>',)</code> <code>('<s>', 'i')</code> <code>('i',)</code> <code>('i', 'love')</code> <code>('love',)</code> <code>('love', 'you')</code> <code>('you',)</code> <code>('you', '</s>')</code> <code>('</s>',)</code>	vocabolario	<code>['<s>', 'i', 'love', 'you', '</s>']</code>
Cosa	Valore prodotto									
sequenza originale	<code>['i', 'love', 'you']</code>									
everygram generati (1-grammi + 2-grammi, con padding)	<code>('<s>',)</code> <code>('<s>', 'i')</code> <code>('i',)</code> <code>('i', 'love')</code> <code>('love',)</code> <code>('love', 'you')</code> <code>('you',)</code> <code>('you', '</s>')</code> <code>('</s>',)</code>									
vocabolario	<code>['<s>', 'i', 'love', 'you', '</s>']</code>									

		concatenato per ciascuna delle 10.000 sequenze in <code>sequenze_tweet</code> .
127	<code>train_lt_mle, vocab_lt_mle = padded_everygram_pipeline(n, sequenze_lett)</code>	Stessa cosa per il dominio letterario, partendo dalle sequenze per-frase.
128-129	<code>modello_tweet = MLE(n) modello_tweet.fit(train_tw_mle, vocab_tw_mle)</code>	Istanza un modello MLE di ordine 2 e lo addestra: <code>.fit()</code> consuma i generatori, costruisce le tabelle di conteggio degli n-grammi e il vocabolario interno del modello. Dopo questa chiamata, i generatori <code>train_tw_mle / vocab_tw_mle</code> sono esauriti (non riutilizzabili).
130-131	<code>modello_lett = MLE(n) modello_lett.fit(train_lt_mle, vocab_lt_mle)</code>	Stessa procedura per il modello letterario MLE.
133	# rigenerata per Laplace: i generatori Python si esauriscono dopo il primo uso	Perché non si riusano <code>train_tw_mle / vocab_tw_mle</code> : in Python i generatori sono iterabili "usa e getta" — una volta scorsi da <code>.fit()</code> , un secondo tentativo di iterarli non produce più elementi. Per addestrare un secondo modello (Laplace) sugli stessi dati occorre quindi richiamare <code>padded_everygram_pipeline</code> da capo, per ottenere generatori "freschi".
134-135	<code>train_tw_lp, vocab_tw_lp = padded_everygram_pipeline(n, sequenze_tweet) train_lt_lp, vocab_lt_lp = padded_everygram_pipeline(n, sequenze_lett)</code>	Rigenera la pipeline di training/vocabolario, identica nella logica alle righe 126–127 ma con nomi di variabile distinti per non sovrascrivere i generatori già usati (anche se già esauriti, tenerli distinti rende il codice più leggibile).
137	<code>modello_tweet_lp = Laplace(n) # Laplace: aggiunge +1 fittizio a ogni conteggio (smoothing)</code>	Il modello <code>Laplace</code> applica l'add-one smoothing: alla formula MLE $P(w c) = \text{count}(c,w) / \text{count}(c)$ viene sostituita $P(w c) = (\text{count}(c,w)+1) / (\text{count}(c)+ V)$. In pratica riserva una piccola probabilità anche a combinazioni mai osservate in training, cosa che MLE non può fare (assegnerebbe probabilità zero).
138	<code>modello_tweet_lp.fit(train_tw_lp, vocab_tw_lp)</code>	Addestra il modello Laplace per Twitter, analogamente a quanto fatto per MLE.
139-140	<code>modello_lett_lp = Laplace(n) modello_lett_lp.fit(train_lt_lp, vocab_lt_lp)</code>	Istanza e addestra il modello Laplace per il dominio letterario.
142-143	<code>print(f"Vocabolario Twitter: {len(modello_tweet.vocab)} parole") # include i token di padding print(f"Vocabolario Letterario: {len(modello_lett.vocab)} parole")</code>	Stampa la dimensione del vocabolario <i>interno al modello</i> (diverso da <code>vocab_tweet_set</code> calcolato prima): include anche i token speciali <code><s></code> , <code></s></code> e <code><UNK></code> aggiunti dalla pipeline di padding, da cui il commento di chiarimento.

7. Probabilità condizionali con MLE (righe 145–156)

Si interroga direttamente il modello con `.score()` per verificare, con esempi concreti, che ciascun modello abbia effettivamente "imparato" le regolarità tipiche del proprio dominio.

Riga	Codice	Spiegazione
145	<pre># .score(parola, [contesto]) = P(parola contesto), stimata dalle frequenze osservate</pre>	<code>score(word, context)</code> è il metodo dei modelli <code>nltk.lm</code> che restituisce la probabilità condizionale stimata. Il contesto va passato come lista (qui di un solo elemento, coerente con un modello bigramma).
148–151	<pre>print(f" P('love' 'i') = {modello_tweet.score('love', ['i']):.4f}") print(f" P('wait' 'cant') = {modello_tweet.score('wait', ['cant']):.4f}") print(f" P('happy' 'so') = {modello_tweet.score('happy', ['so']):.4f}") print(f" P('mr' 'said') = {modello_tweet.score('mr', ['said']):.4f}")</pre>	Sul modello Twitter vengono interrogate tre coppie tipiche del gergo social ("i love", "cant wait", "so happy") che ci si aspetta abbiano probabilità non trascurabile, e una coppia tipicamente letteraria ("mr said") che invece dovrebbe risultare rara o assente in questo dominio — un test di "specificità" del modello.
152–156	<pre>print("\nModello LETTERARIO:") print(f" P('mr' 'said') = {modello_lett.score('mr', ['said']):.4f}") print(f" P('been' 'had') = {modello_lett.score('been', ['had']):.4f}") print(f" P('love' 'i') = {modello_lett.score('love', ['i']):.4f}") print(f" P('wait' 'cant') = {modello_lett.score('wait', ['cant']):.4f}")</pre>	Speculare sul modello letterario: qui "mr said" e "had been" sono attese come frequenti (tipiche del romanzo ottocentesco), mentre "i love"/"cant wait" dovrebbero risultare rare o nulle, dimostrando per contrasto la specializzazione di dominio di ciascun modello.

8. Generazione di testo con MLE (righe 158–191)

I modelli n-gram non servono solo a calcolare probabilità: possono anche **generare** nuovo testo campionando parola dopo parola secondo le probabilità apprese. Qui si genera testo con entrambi i modelli per "vedere" concretamente lo stile che hanno imparato.

Riga	Codice	Spiegazione																		
160-161	<pre>def genera_testo(modello, n_parole=30): """Campiona n_parole token dal modello e rimuove gli artefatti tipici dei bigrammi (padding, lettere isolate)."""</pre>	Funzione di utilità che incapsula generazione + pulizia dell'output, per non ripetere questa logica ogni volta che si vuole stampare del testo generato.																		
162	<pre>testo_raw = modello.generate(n_parole, random_seed=42)</pre>	<code>.generate()</code> campiona <code>n_parole</code> token in sequenza, scegliendo ogni parola in base alla distribuzione di probabilità condizionata dalla parola precedente. Perché <code>random_seed=42</code> : fissare il seed rende la generazione riproducibile — ogni esecuzione dello script produce esattamente lo stesso output, utile sia per il debug sia per confronti stabili tra domini. Cosa succede internamente, iterazione per iterazione (traccia reale ottenuta interrogando <code>modello_tweet.generate(20, random_seed=42)</code> prima di qualunque filtro): il modello parte dal contesto <code><s></code> (inizio sequenza) e sceglie la prima parola campionando dalla distribuzione $P(w \mid \langle s \rangle)$; ottiene <code>"my"</code> . Poi usa <code>"my"</code> come nuovo contesto e campiona $P(w \mid \text{"my"})$, ottenendo <code>"app"</code> ; e così via, una parola alla volta, sempre condizionando solo sull'ultima parola generata (coerente con l'assunzione di Markov di ordine 1). L'output grezzo risultante è: <table><tr><th>#</th><th>Token generato</th></tr><tr><td>0</td><td>my</td></tr><tr><td>1</td><td>app</td></tr><tr><td>2</td><td>im</td></tr><tr><td>3</td><td>glad</td></tr><tr><td>4</td><td>you</td></tr><tr><td>5</td><td>maybe</td></tr><tr><td>6</td><td>u</td></tr><tr><td>7</td><td></s></td></tr></table>	#	Token generato	0	my	1	app	2	im	3	glad	4	you	5	maybe	6	u	7	</s>
#	Token generato																			
0	my																			
1	app																			
2	im																			
3	glad																			
4	you																			
5	maybe																			
6	u																			
7	</s>																			

8	great
9	</s>
10	and
11	makes
12	doesnt
13	even
14	order
15	it
16	being
17	sour
18	kind
19	</s>

Cosa succede quando esce </s> "in mezzo"
(posizioni 7, 9, 19): siccome ogni tweet era una sequenza di addestramento indipendente (righe 70–72), il modello ha imparato anche i bigrammi di confine (parola, </s>) e (</s>, parola) (vedi l'esempio della riga 126). Quando il campionamento sceglie </s>, il modello lo tratta come una parola qualunque e continua a generare usandolo come contesto — $P(w \mid \text{</s>})$ è la distribuzione delle parole che aprono un tweet. Il risultato pratico è che `.generate()` concatena, senza saperlo, pezzi di più "tweet virtuali" diversi in un'unica sequenza continua.

```

164-170 testo_filtrato = []
        for token in testo_raw:
            if token in ['<s>', '</s>',
                '<UNK>']:
                continue
            if len(token) == 1 and
token not in ['i', 'a']:
                continue

        testo_filtrato.append(token)

```

Due filtri applicati token per token: (1) scarta i marcatori tecnici di padding e la parola sconosciuta, che non fanno parte del testo "vero" e comparirebbero come rumore nell'output stampato; (2) scarta i token di un solo carattere *tranne* "i" e "a", che sono le uniche parole inglesi di una lettera valide — gli altri caratteri isolati sono tipicamente artefatti della tokenizzazione (es. apostrofi separati) o del campionamento su un modello povero di contesto come un bigramma.

Traccia reale del ciclo sui 20 token dell'esempio sopra — questa è esattamente l'esecuzione, iterazione per iterazione, di questo `for` :

token	test riga 166 (marcatore ?)	test riga 168 (carattere isolato?)	esito
'my'	no	no (4 caratteri)	TENUTO
'app'	no	no	TENUTO

		<table><tr><td>'im'</td><td>no</td><td>no</td><td>TENUTO</td></tr><tr><td>'glad'</td><td>no</td><td>no</td><td>TENUTO</td></tr><tr><td>'you'</td><td>no</td><td>no</td><td>TENUTO</td></tr><tr><td>'maybe'</td><td>no</td><td>no</td><td>TENUTO</td></tr><tr><td>'u'</td><td>no</td><td>sì (1 carattere , non è "i"/"a")</td><td>SCARTATO</td></tr><tr><td>'</s>'</td><td>sì</td><td>– (già scartato dal primo test)</td><td>SCARTATO</td></tr><tr><td>'great'</td><td>no</td><td>no</td><td>TENUTO</td></tr><tr><td>'</s>'</td><td>sì</td><td>–</td><td>SCARTATO</td></tr><tr><td>... (10 token restanti)</td><td>no</td><td>no</td><td>tutti TENUTI</td></tr></table> Risultato dopo il filtro (riga 172, ' '.join(...)): "my app im glad you maybe great and makes doesnt even order it being sour kind" — esattamente l'inizio della stringa che lo script stampa realmente come "Stile TWITTER". Da notare il token 'u' scartato: è l'abbreviazione informale di "you" tipica di Twitter, ma essendo di un solo carattere viene eliminato dal filtro insieme ai veri artefatti — un piccolo effetto collaterale non voluto della regola "scarta i token di lunghezza 1".	'im'	no	no	TENUTO	'glad'	no	no	TENUTO	'you'	no	no	TENUTO	'maybe'	no	no	TENUTO	'u'	no	sì (1 carattere , non è "i"/"a")	SCARTATO	'</s>'	sì	– (già scartato dal primo test)	SCARTATO	'great'	no	no	TENUTO	'</s>'	sì	–	SCARTATO	... (10 token restanti)	no	no	tutti TENUTI
'im'	no	no	TENUTO																																			
'glad'	no	no	TENUTO																																			
'you'	no	no	TENUTO																																			
'maybe'	no	no	TENUTO																																			
'u'	no	sì (1 carattere , non è "i"/"a")	SCARTATO																																			
'</s>'	sì	– (già scartato dal primo test)	SCARTATO																																			
'great'	no	no	TENUTO																																			
'</s>'	sì	–	SCARTATO																																			
... (10 token restanti)	no	no	tutti TENUTI																																			
172	<code>return ' '.join(testo_filtrato)</code>	Ricomponi i token superstiti in una stringa leggibile, separata da spazi.																																				
174-178	<pre>print("\nStile TWITTER:") print(f" {genera_testo(modello_tweet, 40)}") print("\nStile LETTERARIO:") print(f" {genera_testo(modello_lett, 40)}")</pre>	Genera e stampa 40 parole per ciascun modello, per un confronto diretto e qualitativo dello "stile" appreso dai due modelli.																																				
180-181	<pre># seed diversi = campioni diversi dello stesso stile print("\nConfronto generazione per dominio:")</pre>	Motiva il ciclo seguente: usare più seed permette di osservare più realizzazioni diverse dello stesso modello, mostrando sia la variabilità del campionamento sia la coerenza di fondo dello stile (parole/coppie tipiche del dominio dovrebbero ricomparire anche cambiando seed).																																				

182-184	<pre> for seed in [42, 99, 7, 123]: tw_raw = modello_tweet.generate(15, random_seed=seed) lt_raw = modello_lett.generate(15, random_seed=seed) </pre>	Itera su quattro semi diversi, generando 15 parole da ciascun modello per ogni seed (testi più brevi rispetto a prima, essendo solo un confronto di stile e non un output "in bella").
186-187	<pre> tw_filt = [t for t in tw_raw if t not in ['<s>', '</s>', '<UNK>'] and not (len(t)==1 and t not in ['i', 'a'])] lt_filt = [t for t in lt_raw if t not in ['<s>', '</s>', '<UNK>'] and not (len(t)==1 and t not in ['i', 'a'])] </pre>	Stessa logica di filtraggio di <code>genera_testo</code>, ma scritta inline come list comprehension invece di richiamare la funzione. È una duplicazione di codice (si sarebbe potuto riusare <code>genera_testo</code> con un seed parametrico), presente probabilmente perché qui servono anche i token grezzi per eventuali usi intermedi; funzionalmente equivalente.
189-191	<pre> print(f"\n seed={seed}") print(f" TWITTER: {' '.join(tw_filt)}") print(f" LETTERARIO: {' '.join(lt_filt)}") </pre>	Stampa affiancata dei due stili per lo stesso seed, per facilitarne il confronto diretto riga per riga nell'output.

9. Perplexity cross-domain (righe 193–239)

La **perplexity** misura quanto un modello "si aspetta" una certa sequenza di parole: più è bassa, più il testo è prevedibile per quel modello. Qui si usa questa metrica per costruire un piccolo classificatore di dominio: si assegna una frase al modello (Twitter o Letterario) sotto cui risulta meno "sorprendente".

Riga	Codice	Spiegazione										
195–197	<pre>def calcola_perplexity(modello, tokens, n=2): """perplexity() si aspetta n-grammi (tuple), non token grezzi: senza questa conversione indicizza per errore i caratteri delle stringhe invece di calcolare probabilita' reali."""</pre>	Motivazione della funzione: il metodo <code>model.perplexity()</code> di NLTK non accetta una lista di parole, ma un iterabile di tuple di n token (gli n-grammi già formati). Passargli direttamente una lista di stringhe farebbe sì che internamente NLTK tratti ogni stringa come una sequenza iterabile di caratteri, calcolando un risultato completamente errato senza sollevare eccezioni — un bug silenzioso che questa funzione previene incapsulando la conversione corretta.										
198	<pre> ngr = list(ngrams(pad_both_ends(tokens, n=n), n))</pre>	<code>pad_both_ends(tokens, n=n)</code> aggiunge i marcatori <code><s> / </s></code> alla sequenza di token (coerentemente con come sono stati addestrati i modelli), poi <code>ngrams(..., n)</code> la trasforma nella sequenza di tuple di lunghezza n richiesta da <code>.perplexity()</code> . <code>list()</code> materializza il generatore, perché verrà passato a un metodo che potrebbe doverlo scorrere più di una volta.										
199	<pre> return modello.perplexity(ngr)</pre>	Calcola e restituisce la perplexity del modello sulla sequenza di n-grammi così ottenuta. Esempio reale chiamando <pre>calcola_perplexity(modello_tweet_lp, ["i", "love", "you"], 2) :</pre><table><tr><th>Passo</th><th>Valore</th></tr><tr><td>token originali</td><td>['i', 'love', 'you']</td></tr><tr><td>dopo <code>pad_both_ends(tokens, n=2)</code></td><td>['<s>', 'i', 'love', 'you', '</s>']</td></tr><tr><td>dopo <code>ngrams(..., 2)</code> (input reale a <code>.perplexity()</code>)</td><td>[('<s>', 'i'), ('i', 'love'), ('love', 'you'), ('you', '</s>')]</td></tr><tr><td><code>modello.perplexity(ngr)</code></td><td>53.93</td></tr></table> Da 3 parole si ottengono 4 bigrammi (grazie al padding, righe 44–48 dell'esempio precedente): il modello valuta	Passo	Valore	token originali	['i', 'love', 'you']	dopo <code>pad_both_ends(tokens, n=2)</code>	['<s>', 'i', 'love', 'you', '</s>']	dopo <code>ngrams(..., 2)</code> (input reale a <code>.perplexity()</code>)	[('<s>', 'i'), ('i', 'love'), ('love', 'you'), ('you', '</s>')]	<code>modello.perplexity(ngr)</code>	53.93
Passo	Valore											
token originali	['i', 'love', 'you']											
dopo <code>pad_both_ends(tokens, n=2)</code>	['<s>', 'i', 'love', 'you', '</s>']											
dopo <code>ngrams(..., 2)</code> (input reale a <code>.perplexity()</code>)	[('<s>', 'i'), ('i', 'love'), ('love', 'you'), ('you', '</s>')]											
<code>modello.perplexity(ngr)</code>	53.93											

		quanto è "prevedibile" ciascuna delle 4 transizioni <code><s>→i</code> , <code>i→love</code> , <code>love→you</code> , <code>you→</s></code> , poi le combina in un unico numero di perplexity.		
201-210	<pre>frasi_test = { "tw_pos_1": "i love you so much thank you", ... "lett_4": "he had been very much in love with her", }</pre>	Dizionario di 8 frasi scritte a mano, 4 tipicamente "da tweet" (chiavi che iniziano per <code>tw_</code>) e 4 tipicamente letterarie (chiavi che iniziano per <code>lett_</code>). Il prefisso della chiave funge da etichetta "vera" (ground truth) per valutare poi l'accuratezza del classificatore.		
212-213	<pre>print(f"\n{'Frase':<12} {'PP Twitter':>10} {'PP Lett.':>8} Verdetto") print("-" * 55)</pre>	Intestazione di una tabella testuale formattata con allineamento a colonne fisse, seguita da una riga separatrice.		
215-216	<pre>corretti = 0 totale = 0</pre>	Contatori per calcolare, al termine del ciclo, l'accuratezza complessiva del classificatore basato su perplexity.		
218-219	<pre>for nome, frase in frasi_test.items(): tokens_frase = word_tokenize(frase)</pre>	Itera su ogni frase di test, tokenizzandola per prepararla al calcolo della perplexity.		
220-222	<pre># perplexity bassa = frase piu prevedibile per quel dominio pp_tw = calcola_perplexity(modello_tweet_lp , tokens_frase, n) pp_lett = calcola_perplexity(modello_lett_lp, tokens_frase, n)</pre>	Calcola la perplexity della stessa frase sotto entrambi i modelli. Perché i modelli Laplace e non MLE: le frasi di test contengono quasi certamente bigrammi mai visti esattamente in training (soprattutto nel dominio "sbagliato"); con MLE questi avrebbero probabilità zero, rendendo la perplexity infinita e impossibile da confrontare numericamente. Il modello Laplace, grazie allo smoothing, restituisce sempre un valore finito.		
224-226	<pre># normalizzazione empirica per vocabolario (la perplexity e' gia' normalizzata # per token: dividere anche per V non e' prassi standard, ma qui serve a rendere # confrontabili domini con vocabolari di dimensione molto diversa)</pre>	Commento che segnala onestamente un compromesso metodologico: la perplexity è già una media geometrica per token (quindi già "normalizzata" per la lunghezza della frase), ma i due modelli hanno vocabolari di taglia molto diversa (il corpus Twitter è molto più grande di quello letterario), il che di per sé altera il range di valori di perplexity ottenibile da ciascun modello. Dividere anche per <code> V </code> non è una pratica standard in letteratura, ma qui è un accorgimento pragmatico per rendere i due punteggi comparabili nello stesso esperimento.		
227-228	<pre>pp_tw_n = pp_tw / len(modello_tweet_lp.vocab) pp_lett_n = pp_lett / len(modello_lett_lp.vocab)</pre>	Applica la normalizzazione appena descritta, dividendo ciascuna perplexity per la dimensione del vocabolario del rispettivo modello. Prima iterazione reale del ciclo (nome="tw_pos_1" , frase="i love you so much thank you" , eseguendo davvero lo script): <table><tr><th>Variabile</th><th>Valore</th></tr></table>	Variabile	Valore
Variabile	Valore			

		<table><tr><td>pp_tw (perplexity grezza, modello Twitter)</td><td>128.1</td></tr><tr><td>pp_lett (perplexity grezza, modello Letterario)</td><td>538.7</td></tr><tr><td>len(modello_tweet_lp.vocab)</td><td>12027</td></tr><tr><td>len(modello_lett_lp.vocab)</td><td>9285</td></tr><tr><td>pp_tw_n = 128.1 / 12027</td><td>0.010651</td></tr><tr><td>pp_lett_n = 538.7 / 9285</td><td>0.058018</td></tr><tr><td>verdetto (riga 231): pp_tw_n < pp_lett_n ?</td><td>sì → verdetto="TW"</td></tr><tr><td>verdetto_atteso (da "tw_pos_1")</td><td>"TW"</td></tr><tr><td>corretto? → corretti, totale</td><td>✓ → corretti=1, totale=1</td></tr></table> Da notare: qui la perplexity <i>grezza</i> di Twitter (128.1) è già più bassa di quella letteraria (538.7) — la normalizzazione per vocabolario non cambia il verdetto in questo caso, ma nella riga 158 lett_1 succede l'opposto interessante: pp_tw grezza (6321.1) è molto più alta di pp_lett (290.1), quindi lì il verdetto è "LT" già prima di qualunque normalizzazione. In tutte le 8 frasi del test, verdetto grezzo e verdetto normalizzato coincidono (8/8 = 100% di accuratezza, riga 164 dell'output reale) — segno che, con questi due corpora così diversi per dimensione, la normalizzazione empirica non altera le decisioni finali ma è comunque un'assunzione da verificare caso per caso.	pp_tw (perplexity grezza, modello Twitter)	128.1	pp_lett (perplexity grezza, modello Letterario)	538.7	len(modello_tweet_lp.vocab)	12027	len(modello_lett_lp.vocab)	9285	pp_tw_n = 128.1 / 12027	0.010651	pp_lett_n = 538.7 / 9285	0.058018	verdetto (riga 231): pp_tw_n < pp_lett_n ?	sì → verdetto="TW"	verdetto_atteso (da "tw_pos_1")	"TW"	corretto? → corretti, totale	✓ → corretti=1, totale=1
pp_tw (perplexity grezza, modello Twitter)	128.1																			
pp_lett (perplexity grezza, modello Letterario)	538.7																			
len(modello_tweet_lp.vocab)	12027																			
len(modello_lett_lp.vocab)	9285																			
pp_tw_n = 128.1 / 12027	0.010651																			
pp_lett_n = 538.7 / 9285	0.058018																			
verdetto (riga 231): pp_tw_n < pp_lett_n ?	sì → verdetto="TW"																			
verdetto_atteso (da "tw_pos_1")	"TW"																			
corretto? → corretti, totale	✓ → corretti=1, totale=1																			
230	<pre>verdetto_atteso = "TW" if nome.startswith("tw") else "LT"</pre>	Ricava l'etichetta "vera" dalla chiave del dizionario (vedi riga 201), usata come riferimento per valutare la correttezza della predizione.																		
231	<pre>verdetto = "TW" if pp_tw_n < pp_lett_n else "LT"</pre>	Regola di classificazione: vince il modello con perplexity <i>normalizzata più bassa</i> , cioè quello per cui la frase risulta più prevedibile/naturale.																		
232-235	<pre>corretto = "✓" if verdetto == verdetto_atteso else "X" if verdetto == verdetto_atteso: corretti += 1 totale += 1</pre>	Confronta la predizione con l'etichetta attesa, sceglie il simbolo grafico per la tabella e aggiorna i contatori di accuratezza.																		

237	<pre>print(f"{nome:<12} {pp_tw:>10.1f} {pp_lett:>8.1f} {verdetto} {corretto}")</pre>	Stampa la riga della tabella per questa frase, mostrando le perplexity non normalizzate (i valori grezzi sono più interpretabili in output, mentre la normalizzazione serviva solo internamente per la decisione).
239	<pre>print(f"\nAccuratezza classificatore: {corretti}/{totale} = {corretti/totale*100:.0f}%")</pre>	Riepiloga l'accuratezza complessiva del classificatore a perplexity su tutte le frasi di test, come singola metrica di sintesi.

10. Classificatore basato su log-likelihood (righe 241–289)

Viene proposto un secondo classificatore di dominio, alternativo a quello basato su perplexity, che calcola direttamente la log-verosimiglianza media per token. Il codice stesso segnala che questo approccio è ritenuto "più stabile" della perplexity.

Riga	Codice	Spiegazione
242	<pre>print("Log-likelihood: più alta = il modello conosce meglio questo testo\n")</pre>	Chiarisce la direzione della metrica: a differenza della perplexity (dove "meno è meglio"), con la log-likelihood "più è meglio" (i valori sono negativi perché log di probabilità <1, quindi "più alto" significa "meno negativo", cioè più vicino a zero).
244–247	<pre>def log_likelihood(modello, tokens): """Log-probabilità media per token (somma dei log dei bigrammi, divisa per il numero di transizioni): più alta (meno negativa) = modello migliore. Richiede un modello con smoothing (Laplace): con MLE ogni bigramma mai visto avrebbe probabilita' 0 e dominerebbe la somma tramite il backoff sotto."""</pre>	Il docstring anticipa due punti chiave: (1) la metrica è la media dei log delle probabilità di transizione, non la somma grezza, per renderla confrontabile fra frasi di lunghezza diversa; (2) il modello deve essere Laplace (o comunque con smoothing) perché con MLE una singola transizione mai vista in training darebbe probabilità $0 \rightarrow \log(0)$ non definito, che verrebbe rimpiazzato dal valore di backoff arbitrario (riga 259), dominando artificialmente il risultato finale.
248	<pre>score = 0.0</pre>	Accumulatore della somma dei log-probabilità, inizializzato a zero.
250	<pre># catena di Markov di ordine 1: ogni parola dipende solo dalla precedente</pre>	Inquadra teoricamente il modello bigramma: è una catena di Markov del primo ordine, l'assunzione semplificativa per cui $P(\text{parola}_i \mid \text{tutta la storia precedente}) \approx P(\text{parola}_i \mid \text{parola}_{i-1})$.
251–254	<pre>for i in range(1, len(tokens)): contesto = [tokens[i-1]] parola = tokens[i] prob = modello.score(parola, contesto)</pre>	Scorre i token a partire dal secondo (indice 1, perché il primo token non ha un predecessore su cui condizionare) e per ciascuna coppia (parola precedente, parola corrente) recupera la probabilità condizionale stimata dal modello.
255–256	<pre>if prob > 0: score += math.log(prob)</pre>	Se la probabilità è positiva (il caso normale con un modello Laplace, che non assegna mai zero), ne accumula il logaritmo naturale. Sommare i log invece di moltiplicare le probabilità dirette evita l'underflow numerico che si avrebbe moltiplicando molti numeri piccoli tra loro.

257-259	<pre>else: # backoff di sicurezza: con Laplace non dovrebbe mai attivarsi (score sempre > 0) score += math.log(1e-10)</pre>	Ramo di sicurezza puramente difensivo: con un modello Laplace correttamente addestrato, <code>score()</code> non dovrebbe mai restituire zero, ma il codice si protegge comunque da un eventuale caso limite (es. contesto del tutto assente dal vocabolario) assegnando una probabilità minuscola ma non nulla, evitando un crash per <code>math.log(0)</code> .																																										
260	<pre>return score / (len(tokens) - 1)</pre>	Divide la somma per il numero di transizioni considerate (<code>len(tokens)-1</code>), ottenendo la log-probabilità <i>media</i> per token: questo rende il punteggio comparabile fra frasi di lunghezza diversa (senza la divisione, una frase più lunga avrebbe quasi sempre un punteggio più negativo solo perché somma più termini). Traccia reale, iterazione per iterazione di <code>log_likelihood(modello_tweet_lp, tokens)</code> su <code>tokens = ['i', 'love', 'you', 'so', 'much', 'thank', 'you']</code> (7 token → il ciclo <code>for i in range(1,7)</code> esegue 6 iterazioni): <table><tr><th>i</th><th>contesto</th><th>parola</th><th>P</th><th>log(P)</th><th>somma parz.</th></tr><tr><td>1</td><td>['i']</td><td>'love'</td><td>0.011377</td><td>-4.4762</td><td>-4.4762</td></tr><tr><td>2</td><td>['love']</td><td>'you'</td><td>0.012466</td><td>-4.3848</td><td>-8.8609</td></tr><tr><td>3</td><td>['you']</td><td>'so'</td><td>0.005627</td><td>-5.1802</td><td>-14.0411</td></tr><tr><td>4</td><td>['so']</td><td>'much'</td><td>0.011941</td><td>-4.4278</td><td>-18.4689</td></tr><tr><td>5</td><td>['much']</td><td>'thank'</td><td>0.000082</td><td>-9.4135</td><td>-27.8824</td></tr><tr><td>6</td><td>['thank']</td><td>'you'</td><td>0.021216</td><td>-3.8530</td><td>-31.7354</td></tr></table> Risultato finale: <code>score / (7-1) = -31.7354 / 6 = -5.2892</code>, che è esattamente il valore "Log-lik Twitter: -5.289" stampato dallo script reale per questa frase. Si nota bene dove "succede qualcosa" di interessante: all'iterazione <code>i=5</code>, la transizione <code>'much' → 'thank'</code> ha probabilità molto più bassa (0.000082, cioè circa 140 volte meno probabile delle altre) delle transizioni vicine — è un bigramma raro nel corpus Twitter ("much thank" non è una sequenza tipica; è più comune "thank you" o "so much"), e infatti il suo <code>log(P)=-9.41</code> è il contributo singolarmente più negativo, quello che "penalizza" di più la somma finale.	i	contesto	parola	P	log(P)	somma parz.	1	['i']	'love'	0.011377	-4.4762	-4.4762	2	['love']	'you'	0.012466	-4.3848	-8.8609	3	['you']	'so'	0.005627	-5.1802	-14.0411	4	['so']	'much'	0.011941	-4.4278	-18.4689	5	['much']	'thank'	0.000082	-9.4135	-27.8824	6	['thank']	'you'	0.021216	-3.8530	-31.7354
i	contesto	parola	P	log(P)	somma parz.																																							
1	['i']	'love'	0.011377	-4.4762	-4.4762																																							
2	['love']	'you'	0.012466	-4.3848	-8.8609																																							
3	['you']	'so'	0.005627	-5.1802	-14.0411																																							
4	['so']	'much'	0.011941	-4.4278	-18.4689																																							
5	['much']	'thank'	0.000082	-9.4135	-27.8824																																							
6	['thank']	'you'	0.021216	-3.8530	-31.7354																																							

262-263	<pre>def classifica_loglik(testo, mod_tw, mod_lt): """Assegna il testo al dominio (Twitter o Letterario) il cui modello lo spiega meglio."""</pre>	Funzione che incapsula l'intero flusso di classificazione di un testo: tokenizzazione, calcolo della log-likelihood sotto entrambi i modelli, decisione e stampa diagnostica.
264	<pre>tokens = word_tokenize(testo.lower())</pre>	Minuscolizza esplicitamente il testo in ingresso prima di tokenizzarlo, così da essere coerente con il vocabolario dei modelli (addestrati su testo già minuscolizzato in fase di pulizia).
265-266	<pre>ll_tw = log_likelihood(mod_tw, tokens) ll_lett = log_likelihood(mod_lt, tokens)</pre>	Calcola la log-likelihood media del testo sotto ciascuno dei due modelli passati come argomento.
268	<pre>verdetto = "TWITTER" if ll_tw > ll_lett else "LETTERARIO" # vince il punteggio più alto (meno negativo)</pre>	Regola di decisione: il dominio vincente è quello il cui modello assegna una probabilità media più alta (log-likelihood meno negativa) al testo, cioè quello che "spiega meglio" la frase.
269	<pre>differenza = abs(ll_tw - ll_lett)</pre>	Calcola il margine di confidenza della decisione: una differenza ampia indica una classificazione netta, una differenza piccola indica un testo ambiguo fra i due domini.
271-275	<pre>print(f" Testo: '{testo}'") print(f" Log-lik Twitter: {ll_tw:.3f}") print(f" Log-lik Letterario: {ll_lett:.3f}") print(f" Verdetto: {verdetto} (differenza {differenza:.2f})") print()</pre>	Stampa un piccolo report diagnostico per ogni testo classificato: i due punteggi grezzi, il verdetto finale e il margine di confidenza, seguito da una riga vuota per separare visivamente i risultati dei diversi testi.
277-286	<pre>testi_classificare = ["i love you so much thank you", ... "mr knightley had always been very fond of her",]</pre>	Otto frasi di test scritte a mano (quattro di stile "social", quattro di stile "letterario"), stavolta senza etichette esplicite nel codice: la correttezza va valutata "a occhio" leggendo l'output, dato che il testo stesso rivela chiaramente lo stile atteso.
288-289	<pre>for testo in testi_classificare: classifica_loglik(testo, modello_tweet_lp, modello_lett_lp)</pre>	Applica il classificatore a ciascuna delle otto frasi, usando sempre i modelli Laplace (necessari, come spiegato, per evitare probabilità nulle su n-grammi non visti in training).

11. MLE vs Laplace su una parola mai vista (righe 291–305)

Esperimento mirato a mostrare in modo esplicito il problema pratico di MLE con dati "fuori vocabolario" (out-of-vocabulary, OOV), e come lo smoothing di Laplace lo risolva.

Riga	Codice	Spiegazione
293	# Confronto diretto sullo stesso caso limite: una parola mai vista in training.	Dichiara l'obiettivo del blocco: isolare un singolo caso limite (parola OOV) e osservare come i due modelli lo gestiscono diversamente, invece di dedurlo indirettamente dai risultati di perplexity/log-likelihood visti finora.
294	parola_rara = "xyzword"	Una stringa scelta apposta perché non è una parola inglese reale e quindi ha probabilità (quasi) certa di non comparire in nessuno dei due corpora di training.
295–297	<pre>print(f"Parola mai vista: '{parola_rara}')</pre> <pre>print(f" MLE P = {modello_tweet.score(parola_rara, ['i']):.6f}") # 0 esatto: MLE non generalizza oltre i dati osservati</pre> <pre>print(f" Laplace P = {modello_tweet_lp.score(parola_rara , ['i']):.6f}") # > 0 grazie allo smoothing</pre>	Interroga entrambi i modelli sulla stessa probabilità condizionale $P(\text{"xyzword"} \mid \text{"i"})$. Il modello MLE restituirà esattamente 0.000000 (la formula per frequenza relativa dà 0/qualcosa quando il conteggio del bigramma è zero); il modello Laplace restituirà invece un valore piccolo ma strettamente positivo, grazie al "+1" fittizio aggiunto a ogni conteggio, inclusi quelli mai osservati.
299	# MLE: parola ignota -> probabilità zero -> perplexity infinita	Spiega la conseguenza a catena: se un solo n-gramma della sequenza ha probabilità 0, la perplexity (che nella formula coinvolge l'inverso del prodotto delle probabilità) diverge a infinito, rendendo il punteggio del tutto inutilizzabile per un confronto quantitativo.
300	frase_rara = word_tokenize("i xyzword lol")	Costruisce una frase di test contenente la parola inventata, per verificare concretamente l'effetto sulla perplexity dell'intera frase (non solo su un singolo bigramma isolato).
301	<pre>pp_mle = calcola_perplexity(modello_tweet, frase_rara, n)</pre>	Calcola la perplexity della frase con il modello MLE: ci si aspetta un valore infinito (o comunque un numero enorme/errore), a causa del bigramma con "xyzword" a probabilità zero.
302	<pre>pp_lap = calcola_perplexity(modello_tweet_lp , frase_rara, n) # Laplace regge il calcolo grazie allo smoothing</pre>	Stesso calcolo con il modello Laplace: qui la perplexity resta un numero finito (anche se probabilmente alto, riflettendo comunque una frase "sorprendente" per il modello), a dimostrazione pratica del vantaggio dello smoothing quando il testo può contenere parole non viste in training.
304–305	<pre>print(f"\nPerplexity MLE su frase con parola rara: {pp_mle}")</pre>	Stampa i due risultati fianco a fianco. Da notare la differenza di formattazione: <code>pp_mle</code> viene stampato senza specifica di

```
print(f"Perplexity Laplace su frase  
con parola rara: {pp_lap:.2f}")
```

formato (perché potrebbe essere la stringa `inf`, che non supporta la formattazione decimale `:.2f`), mentre `pp_lap`, essendo garantito un numero finito, viene arrotondato a 2 decimali.

12. Confronto tra ordini $n=1$, $n=2$, $n=3$ (righe 307–323)

Si osserva come la perplexity cambia al crescere dell'ordine del modello (unigrammi, bigrammi, trigrammi), addestrando ogni volta un nuovo modello Laplace **solo sul dominio Twitter** e misurandone la perplexity su una frase tipica Twitter e su una tipica letteraria.

Riga	Codice	Spiegazione
309	<pre>frase_tw = word_tokenize("i love you so much thank you") # tipica frase Twitter</pre>	Frase di test rappresentativa dello stile Twitter, che il modello (addestrato solo su tweet) dovrebbe trovare relativamente prevedibile a qualunque ordine.
310	<pre>frase_lt = word_tokenize("she had been very well disposed") # tipica frase letteraria</pre>	Frase di test rappresentativa dello stile letterario, che invece è "fuori distribuzione" per un modello addestrato solo su tweet: ci si aspetta una perplexity via via più alta.
312	<pre># n crescente = modello piu specifico: la PP letteraria cresce piu in fretta di quella Twitter</pre>	Anticipa il risultato atteso: aumentando l'ordine n , il modello richiede sequenze di contesto sempre più lunghe e specifiche per aver "visto" qualcosa di simile in training; poiché il training è fatto solo su tweet, la frase letteraria (stilisticamente diversa) diventerà sempre più imprevedibile molto più rapidamente della frase Twitter (che è nel dominio di addestramento).
313	<pre>for n_test in [1, 2, 3]:</pre>	Itera sui tre ordini di modello da confrontare: unigramma (nessun contesto), bigramma, trigramma.
314	<pre>tr_tmp, voc_tmp = padded_everygram_pipeline(n_test, sequenze_tweet)</pre>	Ricostruisce da capo la pipeline di addestramento per l'ordine corrente <code>n_test</code> , usando sempre e solo le sequenze Twitter (il confronto è mirato a un solo dominio, non cross-domain come nelle sezioni precedenti).
315	<pre>mod_tmp = Laplace(n_test) # Laplace necessario: la frase letteraria contiene n-grammi mai visti in questo training</pre>	Perché Laplace anche qui: con n che cresce, e con un training set fatto solo di tweet, la probabilità che la frase letteraria contenga trigrammi mai osservati è altissima; senza smoothing (cioè con MLE) la perplexity diventerebbe infinita già a $n=2$ o $n=3$, rendendo impossibile osservare la tendenza crescente che l'esperimento vuole mostrare.
316	<pre>mod_tmp.fit(tr_tmp, voc_tmp)</pre>	Addestra il modello temporaneo sull'ordine corrente.
318–319	<pre>pp_tw = calcola_perplexity(mod_tmp, frase_tw, n_test) pp_lt = calcola_perplexity(mod_tmp, frase_lt, n_test)</pre>	Calcola la perplexity di entrambe le frasi di test sotto il modello di ordine <code>n_test</code> appena addestrato, riusando la funzione di utilità della sezione 9 (che gestisce correttamente padding e formato a tuple).
321–323	<pre>print(f"\nn={n_test}:") print(f" PP su frase Twitter: {pp_tw:.2f}")</pre>	Stampa i risultati per l'ordine corrente, permettendo di leggere in sequenza (per $n=1$, poi $n=2$, poi $n=3$) come

```
print(f" PP su frase  
Letterario: {pp_lt:.2f}")
```

divergono le due perplexity al crescere della complessità del modello.

Le 3 iterazioni reali del ciclo (eseguendo lo script):

n_test	PP frase Twitter	PP frase Letterario	rapporto Lett./Twitter
1 (unigrammi)	124.04	1727.63	≈ 13.9×
2 (bigrammi)	128.14	8059.42	≈ 62.9×
3 (trigrammi)	307.20	10003.72	≈ 32.6×

Si osserva esattamente il fenomeno anticipato dal commento della riga 312: la perplexity sulla frase Twitter resta relativamente contenuta (124→128→307, cresce ma di poco, perché il modello è addestrato su tweet), mentre quella sulla frase letteraria esplode molto più rapidamente (1728→8059→10004), perché a ogni aumento di n il modello richiede sotto-sequenze sempre più lunghe e specifiche che, in un training fatto di soli tweet, semplicemente non compaiono mai in quella forma nel testo letterario.

13. Analisi linguistica: TTR e lunghezza media (righe 325–350)

Ultime due metriche descrittive, indipendenti dai modelli n-gram, per caratterizzare quantitativamente la varietà lessicale e la lunghezza sintattica dei due domini.

Riga	Codice	Spiegazione
327	<pre># TTR = parole uniche / parole totali: più alto = più vario il vocabolario</pre>	Definisce il Type-Token Ratio: il rapporto tra numero di parole distinte ("type") e numero totale di occorrenze ("token"). Un valore vicino a 1 indica poca ripetizione (vocabolario molto vario), un valore vicino a 0 indica alta ripetitività.
328	<pre>N = min(len(tokens_tweet), len(tokens_lett))</pre>	Perché troncare a un campione comune: il TTR è fortemente sensibile alla lunghezza del testo — più token si osservano, più è probabile ripetere parole già viste, quindi il TTR tende naturalmente a scendere all'aumentare della dimensione del campione. Confrontare il TTR di due corpora di dimensioni molto diverse (Twitter ha molti più token del romanzo) senza equalizzare la dimensione del campione produrrebbe un confronto scorretto, in cui il corpus più piccolo risulterebbe artificialmente "più vario" solo perché più corto.
329–330	<pre>ttr_tweet = len(set(tokens_tweet[:N])) / N ttr_lett = len(set(tokens_lett[:N])) / N</pre>	Calcola il TTR usando solo i primi <code>N</code> token di ciascun corpus (lo stesso <code>N</code> per entrambi), rendendo così il confronto equo.
332–338	<pre>print(f"\nType-Token Ratio (su campioni di {N} token):") print(f" Twitter: {ttr_tweet:.4f}") print(f" Letterario: {ttr_lett:.4f}") if ttr_tweet > ttr_lett: print(" → Twitter ha vocabolario più vario (slang, abbreviazioni)") else: print(" → Letterario ha vocabolario più vario a parità di token campionati")</pre>	Stampa i due valori di TTR e un'interpretazione testuale automatica del risultato, scelta dinamicamente in base a quale dei due domini risulta effettivamente più vario nel campione analizzato (il codice non presuppone a priori quale dei due vincerà).
340	<pre># lunghezza media: tweet interi vs frasi del romanzo (sent_tokenize)</pre>	Chiarisce l'unità di confronto: per Twitter l'unità naturale è il tweet intero, per il romanzo è la singola frase (individuata da <code>sent_tokenize</code>) — sono le due unità testuali "atomiche" più sensate per ciascun dominio.
341–342	<pre>lung_tw = [len(word_tokenize(t)) for t in tweets_raw]</pre>	Calcola, per ciascun tweet e ciascuna frase letteraria, il numero di token che lo compongono.

	<pre>lung_lt = [len(word_tokenize(f)) for f in frasi_lett_raw]</pre>	Nota: qui si riusa <code>tweets_raw</code> (testo originale, non ripulito) invece di <code>tweets_puliti</code>: la lunghezza in token viene quindi misurata sul testo "naturale", includendo punteggiatura ed elementi che la pulizia avrebbe rimosso.
344-345	<pre>media_tw = sum(lung_tw) / len(lung_tw) media_lt = sum(lung_lt) / len(lung_lt)</pre>	Calcola la media aritmetica delle lunghezze per ciascun dominio.
347-350	<pre>print(f"\nLunghezza media:") print(f" Tweet: {media_tw:.1f} parole") print(f" Frase letteraria: {media_lt:.1f} parole") print(f" → Frasi letterarie {media_lt - media_tw:.1f} parole più lunghe in media")</pre>	Stampa le due medie e la loro differenza assoluta, quantificando in modo diretto quanto le frasi letterarie tendano a essere sintatticamente più lunghe ed elaborate rispetto ai tweet.

14. Generazione dei grafici riassuntivi (righe 352–458)

Tutti i risultati numerici prodotti finora vengono riassunti visivamente in un'unica figura con quattro pannelli (griglia 2×2), salvata come immagine PNG. L'intero blocco è avvolto in un `try/except` per non far fallire l'intero script se la generazione del grafico incontra un problema.

Riga	Codice	Spiegazione
353	<code>try:</code>	Perché un <code>try/except</code> attorno a tutto il blocco grafico: la generazione di grafici con matplotlib può fallire per motivi ambientali (font mancanti, problemi di rendering, permessi di scrittura sul disco) che non hanno nulla a che vedere con la correttezza dell'analisi linguistica già completata. Isolare questa fase in un blocco protetto garantisce che un problema qui non cancelli né interrompa tutto il lavoro (stampe, statistiche) già svolto e mostrato con successo.
354–355	<code>c_tw = '#2980b9'</code> <code>c_lt = '#e67e22'</code>	Definisce una palette di colori coerente e riutilizzata in tutti e quattro i pannelli: blu per Twitter, arancione per il dominio letterario. Usare le stesse due tinte ovunque rende il grafico immediatamente leggibile: il colore da solo basta a capire a quale dominio si riferisce una barra.
357–359	<code>fig, axes = plt.subplots(2, 2,</code> <code>figsize=(16, 13),</code> <code>facecolor='#f8f9fa')</code> <code>fig.suptitle('Analisi</code> <code>Comparativa N-Gram: Twitter vs</code> <code>Dominio Letterario',</code> <code> fontsize=16,</code> <code>fontweight='bold', color='#1a252f',</code> <code>y=0.98)</code>	<code>plt.subplots(2, 2, ...)</code> crea una figura con una griglia di 4 assi (2 righe × 2 colonne), restituendo <code>axes</code> come matrice 2×2 indicizzabile (<code>axes[riga, colonna]</code>). <code>figsize</code> in pollici determina una figura grande e leggibile una volta esportata; <code>suptitle</code> imposta il titolo generale sopra tutti i sottografici.
361–370	<code># --- Grafico 1: top bigrammi</code> <code>Twitter ---</code> <code>top_tw =</code> <code>bi_tweet_filtrati.most_common(10)</code> <code>[::-1] # invertito: il più</code> <code>frequente in alto</code> <code>y_labels_tw = [f"{p[0]} {p[1]}"</code> <code>for p, _ in top_tw]</code> <code>counts_tw = [c for _, c in</code> <code>top_tw]</code> <code>bars_tw =</code> <code>axes[0,0].barh(y_labels_tw,</code> <code>counts_tw, color=c_tw, alpha=0.85,</code> <code>height=0.6)</code> <code>axes[0,0].set_title('Top 10</code>	Pannello in alto a sinistra: grafico a barre orizzontali (<code>barh</code>) dei 10 bigrammi Twitter più frequenti (senza stopword, riusando <code>bi_tweet_filtrati</code> dalla sezione 4). Perché <code>[::-1]</code>: matplotlib disegna le barre orizzontali dal basso verso l'alto nell'ordine in cui riceve i dati; per far apparire il bigramma più frequente <i>in cima</i> al grafico (lettura naturale "dall'alto verso il basso, dal più al meno importante") occorre invertire l'ordine della lista prima di passarla a <code>barh</code> . <code>bar_label</code> scrive il valore numerico accanto a ciascuna barra, per non dover stimare i valori a occhio dalla scala dell'asse.

	<pre> Bigrammi: Twitter\n(Senza Stopwords)', fontsize=12, fontweight='bold', pad=10) axes[0,0].set_xlabel('Frequenza Assoluta', fontsize=10) axes[0,0].grid(axis='x', linestyle='--', alpha=0.5) axes[0,0].bar_label(bars_tw, padding=5, fontsize=9, color='#475569') </pre>	
372-381	<pre> # --- Grafico 2: top bigrammi letterari --- ... (analogo al Grafico 1, su axes[0,1] con bi_lett_filtrati e colore c_lt) </pre>	Pannello in alto a destra: stesso tipo di grafico del pannello 1, ma per il dominio letterario, posizionato nella cella <code>axes[0,1]</code> con il colore arancione <code>c_lt</code>, per un confronto visivo diretto affiancato ai bigrammi Twitter.
383-389	<pre> # --- Grafico 3: log-likelihood a confronto (barre affiancate) --- testi_graf = [("i love you so much", "TW"), ("i miss you feel so sad", "TW"), ("she had been very well", "LT"), ("it was a very good thing", "LT"),] </pre>	Pannello in basso a sinistra: seleziona un piccolo sottoinsieme di 4 frasi (2 per dominio) da ri-classificare con la log-likelihood, stavolta per visualizzarne il punteggio in un grafico invece che solo in stampa testuale come nella sezione 10.
391-399	<pre> ll_tw_vals = [] ll_lett_vals = [] labels_graf = [] for testo, tipo in testi_graf: tok = word_tokenize(testo.lower()) ll_tw_vals.append(log_likelihood(mo dello_tweet_lp, tok)) ll_lett_vals.append(log_likelihood(modello_lett_lp, tok)) labels_graf.append(f" [{tipo}] {testo}") </pre>	Per ciascuna delle 4 frasi, calcola la log-likelihood sotto entrambi i modelli (riusando la funzione della sezione 10) e costruisce tre liste parallele: i punteggi Twitter, i punteggi letterari, e le etichette da mettere sull'asse x (con il prefisso <code>[TW]</code> / <code>[LT]</code> per ricordare il dominio atteso).
401-402	<pre> x = np.arange(len(labels_graf)) w = 0.35 </pre>	<code>np.arange</code> genera le posizioni numeriche di base (0,1,2,3) sull'asse x, una per frase; <code>w</code> è la larghezza di ciascuna barra, usata per calcolare gli spostamenti necessari a disegnare due barre affiancate (non sovrapposte) per ogni gruppo.
404-405	<pre> axes[1,0].bar(x - w/2, ll_tw_vals, w, label='Modello Twitter', color=c_tw, alpha=0.85) axes[1,0].bar(x + w/2, </pre>	Disegna due serie di barre verticali per ciascuna frase: una spostata a sinistra della posizione centrale (<code>x - w/2</code>) per il modello Twitter, una a destra (<code>x + w/2</code>) per il modello

	<pre>ll_lett_vals, w, label='Modello Letterario', color=c_lt, alpha=0.85)</pre>	letterario — la tecnica standard in matplotlib per ottenere "grouped bar chart" leggibili.
407-413	<pre>axes[1,0].set_xticks(x) axes[1,0].set_xticklabels(labels_graf, rotation=12, ha='right', fontsize=9) axes[1,0].set_title('Log-Likelihood Cross-Domain\n(Barre più vicine allo 0 in alto = Modello Vincente)', fontsize=12, fontweight='bold', pad=10) axes[1,0].set_ylabel('Log-Likelihood', fontsize=10) axes[1,0].grid(axis='y', linestyle='--', alpha=0.5) axes[1,0].legend(frameon=True, facecolor='#ffffff', edgecolor='none') axes[1,0].set_ylim(top=0)</pre>	Configura le etichette dell'asse x (leggermente ruotate di 12° per non sovrapporsi, dato che le frasi intere sono lunghe), titolo, etichetta asse y, griglia orizzontale leggera e legenda. <code>set_ylim(top=0)</code> forza il limite superiore dell'asse y a 0: dato che i valori di log-likelihood sono sempre negativi, questo evita che matplotlib lasci uno spazio vuoto sopra lo zero e rende immediato notare visivamente quale barra è "più vicina allo zero" (cioè il modello vincente).
415-419	<pre># --- Grafico 4: TTR e lunghezza media, doppio asse Y --- categorie = ['Twitter (Social)', 'Letterario (Austen)'] ttr_vals = [ttr_tweet, ttr_lett] lung_vals = [media_tw, media_lt] x2 = np.arange(len(categorie))</pre>	Pannello in basso a destra: riassume in un solo grafico le due metriche linguistiche calcolate nella sezione 13 (TTR e lunghezza media), una per categoria di dominio.
422-426	<pre>color_ttr = '#16a085' axes[1,1].bar(x2 - 0.2, ttr_vals, 0.35, label='Type-Token Ratio (TTR)', color=color_ttr, alpha=0.8) axes[1,1].set_ylabel('Richiesta Lessicale (TTR)', color=color_ttr, fontweight='bold') axes[1,1].tick_params(axis='y', labelcolor=color_ttr) axes[1,1].set_ylim(0, max(ttr_vals) * 1.2)</pre>	Disegna le barre del TTR (valori tipicamente fra 0 e 1) sull'asse y primario, colorate di verde-teal; l'etichetta e i tick dell'asse y vengono colorati in tinta per associarli visivamente a questa serie di dati. Il limite superiore viene fissato al 120% del valore massimo, lasciando un margine visivo sopra le barre più alte.
428	<pre>ax4_twin = axes[1,1].twinx() # secondo asse Y: le due metriche hanno scale molto diverse</pre>	Perché un asse gemello: il TTR è un valore compreso fra 0 e 1, mentre la lunghezza media delle frasi è dell'ordine di 10-20 parole — rappresentarli sulla stessa scala renderebbe una delle due serie praticamente invisibile (schiacciata vicino allo zero). <code>twinx()</code> crea un secondo asse y che condivide lo stesso asse x ma ha una propria scala indipendente, sovrapposto al primo.

429-433	<pre> color_lung = '#8e44ad' ax4_twin.bar(x2 + 0.2, lung_vals, 0.35, label='Lunghezza Media Frase', color=color_lung, alpha=0.8) ax4_twin.set_ylabel('Lunghezza Media Frase (Parole)', color=color_lung, fontweight='bold') ax4_twin.tick_params(axis='y', labelcolor=color_lung) ax4_twin.set_ylim(0, max(lung_vals) * 1.2) </pre>	Disegna le barre della lunghezza media sull'asse gemello, colorate di viola e spostate a destra (<code>x2 + 0.2</code> , complementare al <code>-0.2</code> del TTR) in modo che le due serie non si sovrappongano visivamente pur condividendo la stessa categoria sull'asse x.
435-437	<pre> axes[1,1].set_xticks(x2) axes[1,1].set_xticklabels(categorie , fontsize=10, fontweight='bold') axes[1,1].set_title('Metriche Linguistiche Strutturali a Confronto', fontsize=12, fontweight='bold', pad=10) </pre>	Imposta le etichette dell'asse x condiviso (i due nomi di dominio) e il titolo del pannello.
439-441	<pre> lines1, labels1 = axes[1,1].get_legend_handles_labels () lines2, labels2 = ax4_twin.get_legend_handles_labels() axes[1,1].legend(lines1 + lines2, labels1 + labels2, loc='upper center', frameon=True, facecolor='#ffffff') </pre>	Perché non basta <code>.legend()</code> semplice: avendo due assi distinti (<code>axes[1,1]</code> e <code>ax4_twin</code>), ciascuno "conosce" solo le proprie barre; chiamare <code>.legend()</code> su uno solo dei due mostrerebbe solo metà delle voci. Il codice recupera manualmente le handle/etichette da entrambi gli assi e le unisce in un'unica legenda combinata, disegnata su uno dei due assi.
443-448	<pre> for ax in axes.flat: ax.set_facecolor('#ffffff') for spine in ['top', 'right']: ax.spines[spine].set_visible(False) ax.spines['left'].set_color('#bdc3c7') ax.spines['bottom'].set_color('#bdc3c7') </pre>	Rifinitura estetica applicata a tutti e 4 i pannelli in un solo ciclo (<code>axes.flat</code> appiattisce la matrice 2x2 in una sequenza lineare di 4 assi): sfondo bianco per ogni singolo pannello, rimozione dei bordi superiore e destro (stile "open" più moderno e pulito) e colorazione grigio chiaro dei bordi rimanenti, per uno stile grafico complessivo più sobrio rispetto ai bordi neri di default.

450-451	<pre> cartella_script = os.path.dirname(os.path.abspath(__f ile__)) if '__file__' in globals() else '.' percorso_salvataggio = os.path.join(cartella_script, 'confronto_risultati_laboratorio.pn g') </pre>	Perché non un percorso relativo diretto: se lo script venisse lanciato da una directory di lavoro diversa da quella in cui si trova il file <code>.py</code>, un percorso relativo come <code>'confronto_risultati_laboratorio.png'</code> salverebbe l'immagine nel posto sbagliato. Calcolando la cartella a partire da <code>__file__</code> (il percorso del file sorgente stesso), l'immagine viene sempre salvata accanto allo script, indipendentemente da dove viene eseguito il comando. Il controllo <code>if '__file__' in globals()</code> è un fallback difensivo per il raro caso in cui lo script venga eseguito in un contesto dove questa variabile speciale non è definita (ad es. incollato ed eseguito riga per riga in una shell interattiva), usando in quel caso la directory corrente.
453-455	<pre> plt.tight_layout() plt.savefig(percorso_salvataggio, dpi=200, bbox_inches='tight', facecolor=fig.get_facecolor()) print(f"Grafico salvato con successo in: {percorso_salvataggio}") </pre>	<code>tight_layout()</code> ottimizza automaticamente gli spazi fra i sottografici per evitare sovrapposizioni di etichette/titoli. <code>savefig</code> esporta la figura a 200 DPI (buona qualità di stampa/lettura), con <code>bbox_inches='tight'</code> per rimuovere i margini bianchi in eccesso attorno alla figura, e <code>facecolor</code> impostato esplicitamente sullo stesso colore di sfondo della figura (altrimenti matplotlib userebbe di default uno sfondo trasparente/bianco che potrebbe non combaciare con il colore <code>'#f8f9fa'</code> scelto alla riga 357).
457-458	<pre> except Exception as e: print(f"Errore durante la generazione del grafico dello Step 13: {e}") </pre>	Cattura <i>qualsunque</i> eccezione sollevata nel blocco grafico e la trasforma in un messaggio informativo invece di un traceback che interromperebbe lo script. Il compromesso di questa scelta è che un <code>except Exception</code> molto ampio nasconde anche eventuali bug reali nel codice del grafico (non solo problemi ambientali); è una scelta pragmatica ragionevole in uno script di laboratorio dove i grafici sono un "bonus" visivo e non il cuore dell'esperimento.

15. Riepilogo finale (righe 460–478)

Ultimo blocco: un riepilogo testuale compatto di tutti i numeri e le conclusioni chiave dell'esperimento, utile come "executive summary" leggibile senza dover scorrere tutto l'output precedente.

Riga	Codice	Spiegazione
462–464	<pre>print(f"\nDataset:") print(f" Twitter: {len(tweets_raw)} tweet (5000 pos + 5000 neg)") print(f" Letterario: Emma by Jane Austen (1816)")</pre>	Ricorda la composizione dei due dataset di partenza: la nota "(5000 pos + 5000 neg)" è un valore noto a priori della struttura del corpus <code>twitter_samples</code> di NLTK, scritto come testo fisso piuttosto che ricalcolato dinamicamente.
466–468	<pre>print(f"\nVocabolario:") print(f" Twitter: {len(vocab_tweet_set)} parole uniche") print(f" Letterario: {len(vocab_lett_set)} parole uniche")</pre>	Riprende le statistiche di vocabolario calcolate nella sezione 3, riproponendole nel riepilogo finale per comodità di lettura.
470–471	<pre>print(f"\nTTR: Twitter= {ttr_tweet:.4f}, Letterario= {ttr_lett:.4f}") print(f"Lunghezza media: Tweet= {media_tw:.1f}, Letterario= {media_lt:.1f}")</pre>	Riprende in forma compatta le due metriche linguistiche calcolate nella sezione 13.
473–476	<pre>print(f"\nModelli:") print(f" MLE → generazione testo e probabilità condizionali") print(f" Laplace → perplexity (gestisce parole mai viste)") print(f" Log-lik → classificatore (più stabile di perplexity)")</pre>	Sintetizza il "ruolo" attribuito a ciascuna tecnica lungo lo script: MLE per gli usi in cui interessa la probabilità osservata pura (generazione, probabilità condizionali dirette), Laplace ovunque serva un valore finito anche con dati non visti (perplexity), e la log-likelihood come criterio di classificazione preferito rispetto alla perplexity normalizzata (coerentemente con quanto dichiarato nell'intestazione della sezione 10).
478	<pre>print("\nLab completato!")</pre>	Messaggio finale di conferma che l'intero script è arrivato in fondo senza errori bloccanti.

Glossario dei concetti chiave

Riepilogo dei concetti NLP/statistici usati nel codice, per chi vuole un ripasso rapido senza tornare al materiale del corso.

N-gramma

Sequenza di N token consecutivi estratta da un testo. Un bigramma (N=2) cattura dipendenze fra coppie di parole adiacenti, un trigramma (N=3) fra triple.

MLE (Maximum Likelihood Estimation)

Stima della probabilità condizionale come semplice frequenza relativa osservata: $P(w|c) = \text{count}(c,w) / \text{count}(c)$. Non assegna mai probabilità a eventi mai osservati in training (probabilità zero per gli n-grammi "out of vocabulary").

Smoothing di Laplace (add-one)

Tecnica che "sposta" una piccola quantità di probabilità dagli eventi osservati verso quelli non osservati, sommando 1 a ogni conteggio prima di normalizzare: $P(w|c) = (\text{count}(c,w)+1) / (\text{count}(c)+|V|)$. Garantisce che nessun n-gramma abbia mai probabilità esattamente zero.

Padding (<s>, </s>)

Marcatori artificiali di inizio/fine sequenza aggiunti prima dell'addestramento, che permettono al modello di stimare anche la probabilità che una frase *inizi* o *finisca* con una certa parola.

Perplexity

Misura di quanto un modello sia "sorpreso" da una sequenza di testo; formalmente l'inverso della media geometrica delle probabilità assegnate agli n-grammi della sequenza. Valori bassi indicano testo prevedibile per quel modello, valori alti (o infiniti con MLE) indicano testo sorprendente/fuori distribuzione.

Log-likelihood

Somma (qui, media) dei logaritmi delle probabilità condizionali assegnate dal modello a ogni transizione osservata nel testo. Usare i logaritmi evita l'underflow numerico che si avrebbe moltiplicando molte probabilità piccole fra loro.

Type-Token Ratio (TTR)

Rapporto fra parole uniche (type) e parole totali (token) in un testo; misura la varietà lessicale, ma va sempre calcolato su campioni di dimensione uguale per essere confrontabile fra testi diversi.

Catena di Markov di ordine 1

Assunzione semplificativa secondo cui la probabilità di una parola dipende solo dalla parola immediatamente precedente, e non dall'intera storia del testo. È l'assunzione alla base di un modello bigramma.